CS285 HW3 作业报告：Q-Learning 与 Actor-Critic 算法

摘要

本报告围绕 Berkeley CS285 HW3 的三个核心部分展开：第一部分分析多步 Q-learning 中的偏差、方差、on-policy/off-policy 差异以及重要性采样；第二部分实现并实验分析 Deep Q-Network，包括基础 DQN、Double DQN、Atari MsPacman 以及学习率敏感性实验；第三部分实现 Soft Actor-Critic，包括 critic bootstrapping、entropy bonus、REINFORCE actor、reparameterized actor、single-Q/double-Q/clipped double-Q target backup，并在 HalfCheetah、Hopper 与 Humanoid 环境上进行实验。报告按照作业顺序组织，并在实现部分将各个 TODO 的关键代码逐行摘出并添加中文注释。实验结果显示：Q-learning 对学习率非常敏感，过大学习率会导致 critic loss 剧烈震荡；Double DQN 能缓解过估计并提升 LunarLander 稳定性；SAC 中 reparameterization gradient 明显优于 REINFORCE；clipped double-Q 在 Hopper 上取得最高回报且 Q 值更保守，说明其有效抑制了过估计偏差。Q-learning 为什么会不稳定，以及深度强化学习里有哪些办法让它稳定下来

目录

1 Multistep Q-Learning 理论题 **2**

1.1 1.1 TD-Learning Bias 2

1.2 1.2 Tabular Learning 2

1.3 1.3 Variance of Q Estimate 3

1.4 1.4 Function Approximation 3

1.5 1.5 Multistep Importance Sampling 3

2 Deep Q-Learning **5**

2.1 DQN 实现总览 5

2.2 DQN TODO 逐行注释：critic update 5

2.3 DQN TODO 逐行注释：ϵ-greedy action selection 6

2.4 DQN TODO 逐行注释：target network update 6

2.5 DQN TODO 逐行注释：run script 环境交互 7

2.6 CartPole-v1 基础实验与学习率敏感性 8

2.7 Double DQN 实现与 LunarLander-v2 对比 10
2.8 MsPacman-v0 实验 11
2.9 DQN 小结与思考 13

3 Continuous Actions with Actor-Critic: SAC 14
3.1 SAC 实现总览 14
3.2 SAC TODO 逐行注释：run script 交互与 replay buffer 14
3.3 SAC TODO 逐行注释：critic target 与 entropy bonus 15
3.4 SAC TODO 逐行注释：target network soft/hard update 16
3.5 SAC TODO 逐行注释：entropy 计算 16
3.6 SAC TODO 逐行注释：REINFORCE actor loss 16
3.7 SAC TODO 逐行注释：reparameterized actor loss 17
3.8 SAC TODO 逐行注释：single-Q, double-Q, clipped-Q backup 18
3.9 HalfCheetah-v4：actor gradient estimator 对比 18
3.10 Hopper-v4：target backup strategy 对比 19
3.11 Humanoid-v4：最佳配置迁移 21

4 整体讨论与个人思考 22
4.1 从理论到实验的对应关系 22
4.2 为什么 reparameterization 如此重要 22
4.3 我对 DQN 与 SAC 的比较 22

5 结论 22

A 实验结果数值摘要 23

1 Multistep Q-Learning 理论题

1.1 1.1 TD-Learning Bias

题目问：若 $\hat{Q}$ 是 Q 的有噪声但无偏估计，那么 Bellman backup

$$B\hat{Q} = r(s,a) + \gamma \max_{a'} \hat{Q}(s',a')$$

是否是

$$BQ = r(s,a) + \gamma \max_{a'} Q(s',a')$$

的无偏估计？答案是：**No**。

核心原因是 max 是非线性算子。即使每个动作上的 $\hat{Q}(s',a')$ 都是无偏的，取最大值时更容易选中“噪声正好偏大”的动作，因此

$$\mathbb{E}\left[\max_{a'} \hat{Q}(s',a')\right] \neq \max_{a'} \mathbb{E}\left[\hat{Q}(s',a')\right].$$

直观地说，很多个带噪声的估计值放在一起取最大值，相当于给噪声一个“被挑中”的机会。于是 Q-learning 中的 max backup 往往会产生 **overestimation bias**。这也是后面 Double DQN 与 clipped double-Q 要解决的问题。

1.2 1.2 Tabular Learning

三个陈述的含义如下：

- I：$Q_{\phi_{k+1}}$ 是上一轮策略 π_k 的真实 Q 函数 Q^{π_k} 的无偏估计。
- II：当 $k \to \infty$ 且 batch size B 固定时，Q_{ϕ_k} 是 Q^* 的无偏估计。
- III：当 $k, B \to \infty$ 时，可以恢复最优 Q 函数 Q^*。

本题可以用一句话抓住：**bootstrapping 带来偏差，有限数据带来采样误差，off-policy 多步回报若不做修正会产生分布错配**。

表 1: Tabular setting 下 I-III 的判断

设置	I	II	III
$N=1$，on-policy tabular	No	No	Yes
$N=1$，off-policy tabular	No	No	Yes
$N>1$，on-policy tabular	No	No	Yes
$N>1$，off-policy tabular	No	No	No
$N\to\infty$，on-policy tabular	Yes	No	Yes
$N\to\infty$，off-policy tabular	No	No	No

解释如下。对于有限 N，target 中仍然包含 $\max_a Q_{\phi_k}(s,a)$ 这一 bootstrapping 项，因此它不是严格意义上对 Q^{π_k} 的无偏估计。对于固定有限的 B，即使迭代次数无限，采样误差也不会完全消

失，因此 II 不成立。对于 tabular 且有足够数据的情形，$N = 1$ 的 Q-learning 以及 on-policy 的多步控制可以在极限下恢复 Q^*。但是 off-policy 的多步或 Monte Carlo 回报若不进行 importance sampling 修正，就会把行为策略 π' 下的轨迹当成目标策略 π 下的轨迹，因此一般不能保证恢复 Q^*。

1.3 1.3 Variance of Q Estimate

固定数据量 B 时，三种估计的方差排序直觉为：

$$\text{最高方差} : N \to \infty, \qquad \text{最低方差} : N = 1.$$

$N = 1$ 只用一步 reward 加一个 bootstrapped value，因此随机性来源较少，方差最低；但它依赖当前 Q 估计，偏差较大。$N \to \infty$ 退化为 Monte Carlo return，需要把整条轨迹上的随机 reward 都累加进去，方差最高；但它不再依赖 bootstrapped value，偏差较低。这就是 TD 和 Monte Carlo 之间经典的 bias-variance tradeoff。

1.4 1.4 Function Approximation

加入 function approximation 后，情况更脆弱。即使函数类能够表示任意 deterministic policy 的真实 Q^π，有限 N 的 bootstrapping 仍可能因为“拟合误差 + bootstrap 误差 + greedy improvement”产生不稳定。因此最稳妥成立的是：

$$N \to \infty, \quad B \to \infty, \quad k \to \infty$$

时可以收敛到 Q^*。此时没有 bootstrapping，on-policy Monte Carlo 估计在无限数据下可以准确评价当前策略，再通过 greedy improvement 改进策略。

这道题想强调的是：tabular setting 下很多收敛性质依赖“每个状态动作表格项都能被准确更新”；而 function approximation 会引入泛化和拟合误差，理论保证明显变弱。这也是深度强化学习里 DQN/SAC 需要 target network、replay buffer、double Q 等 trick 的原因。

1.5 1.5 Multistep Importance Sampling

若数据来自行为策略 π'，但我们希望估计或改进目标策略 π，则多步回报需要用 likelihood ratio 修正。对从 $t+1$ 到 $t+i$ 的动作序列，importance sampling ratio 为

$$\rho_{t+1:t+i} = \prod_{\ell=t+1}^{t+i} \frac{\pi(a_\ell|s_\ell)}{\pi'(a_\ell|s_\ell)}.$$

修正后的 N-step target 可写为

$$y_{j,t} = r_{j,t} + \sum_{i=1}^{N-1} \gamma^i \rho_{t+1:t+i} r_{j,t+i} + \gamma^N \rho_{t+1:t+N} \max_a Q_{\phi_k}(s_{j,t+N}, a).$$

于是 regression objective 仍是拟合这个修正后的 target：

$$\phi_{k+1} = \arg\min_{\phi \in \Phi} \sum_{j,t} \left(y_{j,t} - Q_\phi(s_{j,t}, a_{j,t})\right)^2.$$

当 $N = 1$ 时，没有中间多步 trajectory action 需要修正，因此通常不需要改动式 (2)。当 $N \to \infty$ 时，整条未来轨迹都来自 π'，off-policy mismatch 最严重，因此 importance sampling 尤其重要。不过这也会带来高方差：乘积型 ratio 很容易爆炸或趋近于 0，这也是实际算法会使用 clipped ratio 或 Retrace 等方法的原因。

Deep Q-Learning

DQN 实现总览

DQN 用神经网络 $Q_\phi(s,a)$ 近似动作价值函数。对一条 transition (s,a,r,s',d)，基础 DQN 的 TD target 为

$$y = r + \gamma(1-d)\max_{a'} Q_{\phi,}(s',a'),$$

其中 $Q_{\phi,}$ 是 target critic。critic loss 是均方 Bellman error：

$$L(\phi) = \mathbb{E}\left[(Q_\phi(s,a) - y)^2\right].$$

Basic Q-Learning 代码完整性补齐

这里单独补齐 Basic Q-learning 的代码实现闭环。作业不是只要求“跑出图”，还明确要求在 `dqn_agent.py` 和 `run_hw3_dqn.py` 中补 TODO。下面按照当前实际代码写法说明每一块在做什么。

`get_action`: ϵ-greedy 动作选择

当前 `dqn_agent.py` 中的实现逻辑如下：

```
observation = ptu.from_numpy(np.asarray(observation))[None]

if np.random.random() < epsilon:
    # 离散动作空间中随机选一个动作；等价于随机探索。
    action = torch.randint(self.num_actions, (1,), device=observation.device)
else:
    # greedy exploitation: 选 Q(s,a) 最大的动作。
    with torch.no_grad():
        qa_values = self.critic(observation)
        action = torch.argmax(qa_values, dim=1)

return ptu.to_numpy(action).squeeze(0).item()
```

这部分已经完成 Basic Q-learning 需要的 ϵ-greedy exploration。训练开始时 exploration schedule 令 ϵ 接近 1，因此基本随机探索；后期 ϵ 下降，agent 更多执行 greedy action。

`update_critic`: 基础 DQN critic update

当前代码先用 target critic 计算下一状态动作值，再根据是否启用 `use_double_q` 选择普通 DQN 或 Double DQN 的 backup。

```
with torch.no_grad():
    # target critic 评估下一状态 s' 的所有动作 Q_target(s', a')。
    next_qa_values = self.target_critic(next_obs)

    if self.use_double_q:
        # Double DQN: online critic 选动作，target critic 估值。
        next_qa_values_online = self.critic(next_obs)
        next_action = torch.argmax(next_qa_values_online, dim=1)
    else:
        # Basic DQN: 直接在 target critic 上取最大动作。
        next_action = torch.argmax(next_qa_values, dim=1)

    # 取 Q_target(s', next_action)。
    next_q_values = torch.gather(
        next_qa_values,
        dim=1,
        index=next_action.unsqueeze(1),
    ).squeeze(1)

    # Bellman target: y = r + gamma * (1-done) * Q_target(s', a')。
    target_values = reward + self.discount * (1 - done.float()) * next_q_values

# online critic 估计当前真实动作 Q(s,a)。
qa_values = self.critic(obs)
q_values = torch.gather(
    qa_values,
    dim=1,
    index=action.long().unsqueeze(1),
).squeeze(1)

loss = self.critic_loss(q_values, target_values)

self.critic_optimizer.zero_grad()
loss.backward()
```

```
if self.clip_grad_norm is not None:
   grad_norm = torch.nn.utils.clip_grad_norm_(
       self.critic.parameters(),
       self.clip_grad_norm,
   )
self.critic_optimizer.step()
self.lr_scheduler.step()
```

这段代码是基础 Q-learning 的核心。普通 DQN 对应 `self.use_double_q=False`; Double DQN 对应 `True`。`with torch.no_grad()` 已经保证 TD target 不会被反向传播污染，因此不需要额外对 `target_values` 写 `detach()`。

`update`: target network hard update

当前 `update` 函数先更新 critic，再按 step 周期同步 target critic。

```
critic_stats = self.update_critic(obs, action, reward, next_obs, done)

if step % self.target_update_period == 0:
    self.update_target_critic()

return critic_stats
```

`update_target_critic()` 内部调用 `load_state_dict`，也就是 hard update：每隔固定步数把 online critic 参数复制给 target critic。这样 TD target 变化更慢，训练更稳定。

`run_hw3_dqn.py`: 环境交互与 replay buffer

当前 DQN run script 的核心循环已经补齐：

```
epsilon = exploration_schedule.value(step)
action = agent.get_action(observation, epsilon=epsilon)

next_observation, reward, done, info = env.step(action)
next_observation = np.asarray(next_observation)
truncated = info.get("TimeLimit.truncated", False)

# TimeLimit 截断不是真正终止，因此 replay buffer 中的 done 应为 False。
replay_done = done and not truncated

if isinstance(replay_buffer, MemoryEfficientReplayBuffer):
    replay_buffer.insert(
        action=action,
        reward=reward,
        next_observation=next_observation[-1, ...],
        done=replay_done,
    )
else:
    replay_buffer.insert(
        observation=observation,
        action=action,
        reward=reward,
        next_observation=next_observation,
        done=replay_done,
    )

if done:
    reset_env_training()
else:
    observation = next_observation

if step >= config["learning_starts"]:
    batch = replay_buffer.sample(config["batch_size"])
    batch = ptu.from_numpy(batch)
    update_info = agent.update(
        batch["observations"],
        batch["actions"],
        batch["rewards"],
        batch["next_observations"],
        batch["dones"],
        step,
    )
```

这里最容易丢分的是 `TimeLimit.truncated`。如果 episode 只是因为最大步数被截断，真实 MDP 并没有终止，所以 TD target 仍应 bootstrap；当前代码已经把这种情况存成 `done=False`，这是对的。

CartPole-v1 基础实验与学习率敏感性

本实验在 CartPole-v1 上比较四个 learning rate：10^{-4}、10^{-3}、10^{-2} 和 5×10^{-2}。结果如图 1。

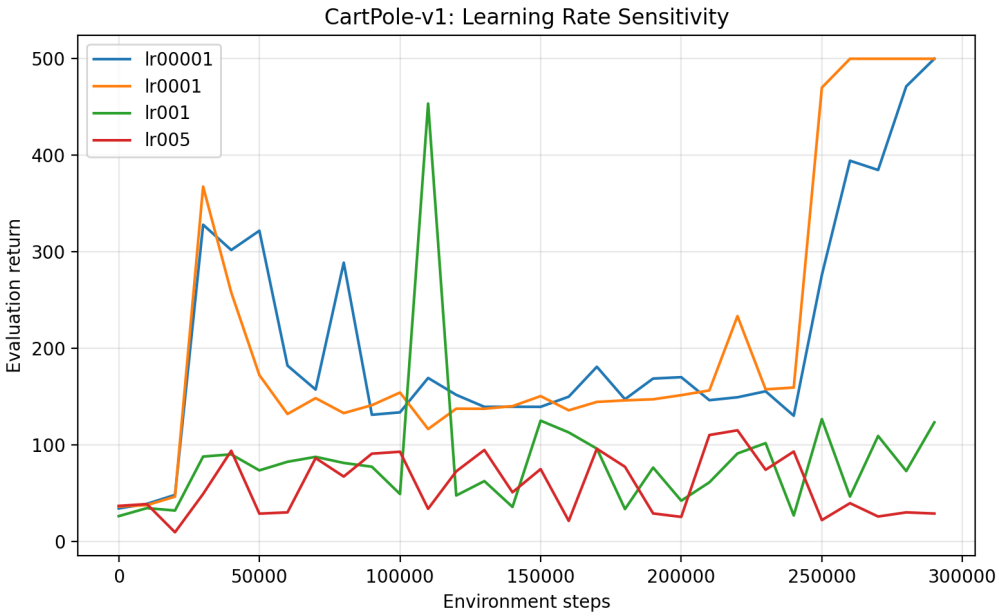
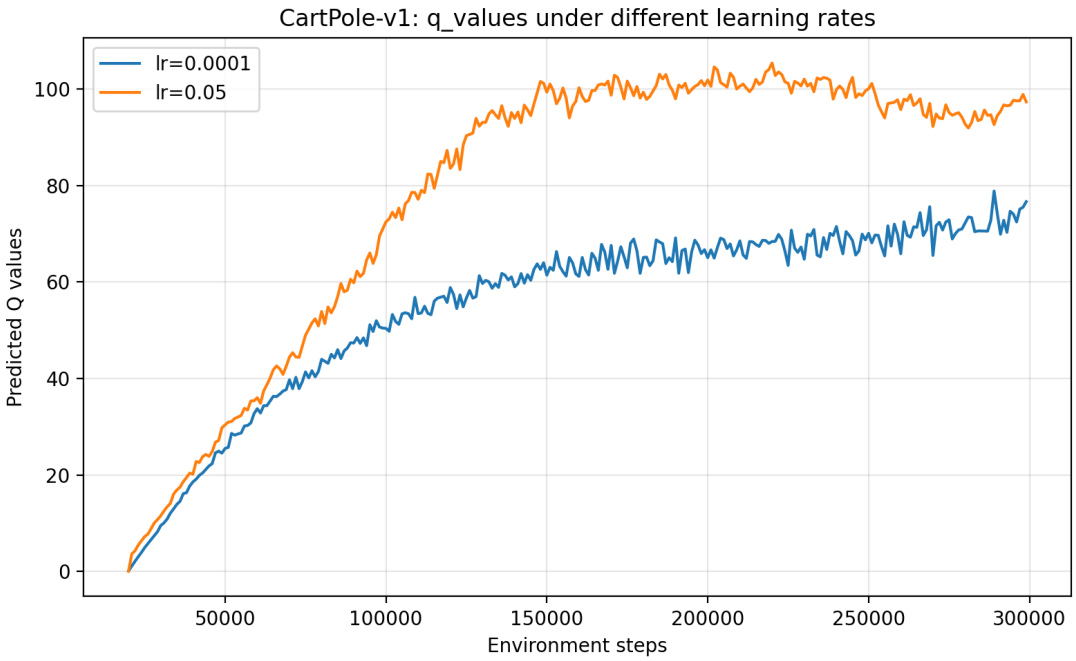


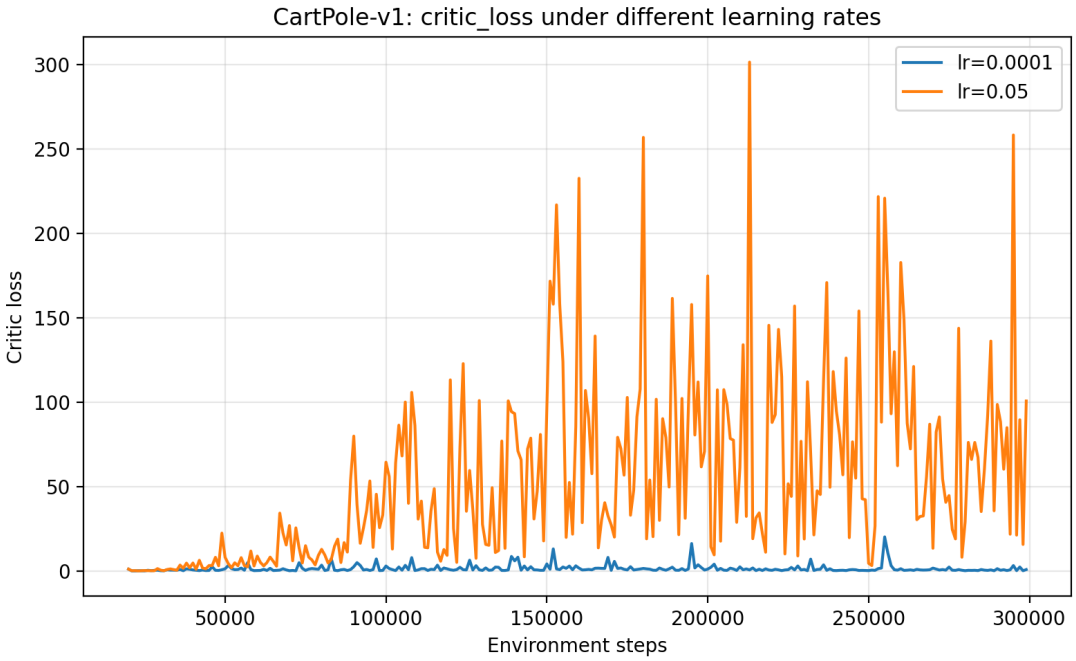
图 1: CartPole-v1 不同学习率下的 evaluation return。

较小学习率最终能达到接近 500 的满分表现，而 5×10^{-2} 表现明显最差，长期停留在较低 return。这说明 DQN 对 learning rate 非常敏感。原因是 DQN 的 target 本身来自网络估计，学习率太大时，一次更新会把 critic 推得过远，使下一轮 target 也剧烈变化，最后形成不稳定的 bootstrapping 过程。

为了进一步说明大学习率的问题，图 2 对比了正常学习率 10^{-4} 与大学习率 5×10^{-2} 下的 predicted Q-values 和 critic loss。



(a) predicted Q-values 对比。



(b) critic loss 对比。

图 2: CartPole-v1 中正常学习率与过大学习率对 Q 值和 critic loss 的影响。

数值上，10^{-4} 的最终 critic loss 约为 0.79，最大约为 20.15；而 5×10^{-2} 的最终 critic loss 约为 100.73，最大超过 300。同时，大学习率下 predicted Q-values 明显更高，最终约 97.36，最大约 105.42，而正常学习率最终约 76.67。这与理论部分的 overestimation bias 和 bootstrapping instability 对应得很清楚。

超参数实验配置文件补齐

作业要求不只是画四条曲线，还要求选择一个超参数并额外跑至少三个设置。本次实际选择的是 learning rate。根据当前代码包和绘图脚本，四个设置应写成：

```
lr = 1e-4
lr = 1e-3  # baseline cartpole.yaml 默认学习率
lr = 1e-2
lr = 5e-2
```

当前代码包里能看到的相关配置包括：

```
experiments/dqn/cartpole.yaml                 # baseline, learning_rate 默认 1e-3
experiments/dqn/cartpole_lr005.yaml           # learning_rate: 0.05
experiments/dqn/my_runs/cartpole_lr0001.yaml  # learning_rate: 0.0001
experiments/dqn/my_runs/cartpole_lr001.yaml   # learning_rate: 0.01
experiments/dqn/my_runs/cartpole_lr005.yaml   # learning_rate: 0.05
```

为了最贴合作业题面的目录要求，最终提交前建议复制或整理成下面这种结构：

```
experiments/dqn/hyperparameters/cartpole_lr_1e-4.yaml
experiments/dqn/hyperparameters/cartpole_lr_1e-3.yaml
experiments/dqn/hyperparameters/cartpole_lr_1e-2.yaml
experiments/dqn/hyperparameters/cartpole_lr_5e-2.yaml
```

选择 learning rate 的原因是：Q-learning 的 target 依赖自身网络估计，学习率过大时会放大 bootstrap target 的移动和 max over noisy estimates 的过估计。因此 learning rate 是观察 DQN 稳定性最直接的超参数。

2 Deep Q-Learning

2.1 DQN 实现总览

DQN 的基本思想是用神经网络 $Q_\phi(s,a)$ 近似动作价值函数，并用 TD target 训练 critic。对一条 transition (s,a,r,s',d)，基础 DQN 的 target 是

$$y = r + \gamma(1-d)\max_{a'} Q_{\phi'}(s',a'),$$

其中 $Q_{\phi'}$ 是 target critic。critic loss 是均方 Bellman error：

$$L(\phi) = \mathbb{E}\left[\left(Q_\phi(s,a) - y\right)^2\right].$$

在实现上，DQN 部分主要完成四类 TODO：critic update、ϵ-greedy action selection、target network update、以及 run script 中环境交互和 replay buffer 采样。下面按 TODO 逻辑逐行摘出并注释。

2.2 DQN TODO 逐行注释：critic update

下面代码对应 `cs285/agents/dqn_agent.py` 中 critic 更新的核心逻辑。

```
# 从 target critic 得到下一状态 s' 上所有离散动作的 Q 值，形状通常是 [batch_size,
↪  num_actions]
next_qa_values = self.target_critic(next_observations)

# 普通 DQN：直接在 target critic 上选择 Q 值最大的动作
next_actions = next_qa_values.argmax(dim=1)

# 如果启用 Double DQN，则动作选择改由 online critic 完成
if self.double_q:
    # online critic 只负责“选动作”，不负责“估值”，从而减少 max 造成的过估计
    next_actions = self.critic(next_observations).argmax(dim=1)

# 根据选出的 next_actions，从 target critic 的输出中取出对应 Q(s', a')
next_q_values = next_qa_values.gather(1, next_actions.unsqueeze(1)).squeeze(1)

# 对终止状态不 bootstrap；如果 done=True，则后一项乘 0
q_targets = rewards + self.gamma * (1 - dones.float()) * next_q_values

# online critic 输出当前状态 s 下所有动作的 Q 值
qa_values = self.critic(observations)

# 从所有动作 Q 值中取出实际执行动作 a 对应的 Q(s, a)
q_values = qa_values.gather(1, actions.long().unsqueeze(1)).squeeze(1)

# 用均方误差拟合 TD target；target detach，避免梯度流入 target 计算图
```

```
loss = self.critic_loss(q_values, q_targets.detach())

# 清空旧梯度，准备本次反向传播
self.optimizer.zero_grad()

# 对 critic loss 反向传播，计算 critic 网络参数梯度
loss.backward()

# 梯度裁剪，防止单次 TD error 太大导致梯度爆炸
torch.nn.utils.clip_grad_norm_(self.critic.parameters(), self.grad_norm_clipping)

# 根据梯度更新 online critic 参数
self.optimizer.step()
```

这段实现里最关键的是两点。第一，target 必须 detach，否则 target network 也会被当前 loss 反向传播污染；第二，Double DQN 的 “选动作” 和 “估值” 分离，能够缓解 max over noisy Q-values 带来的 overestimation bias。

2.3 DQN TODO 逐行注释：ϵ-greedy action selection

下面代码对应 agent 的动作选择逻辑。

```
# 生成一个 [0,1) 之间的随机数，若小于 epsilon，则执行随机探索
if np.random.rand() < epsilon:
    # 从环境动作空间中均匀采样一个动作
    action = self.env.action_space.sample()
else:
    # 把单个 observation 增加 batch 维度，变成 [1, obs_dim]
    obs_tensor = ptu.from_numpy(observation[None])

    # 不需要计算梯度，因为这里只是选择动作，不是训练
    with torch.no_grad():
        # critic 输出当前状态下每个动作的 Q 值
        qa_values = self.critic(obs_tensor)

        # 选择 Q 值最大的动作作为 greedy action
        action = qa_values.argmax(dim=1).item()
```

这个 TODO 的核心是 exploration-exploitation tradeoff。训练初期 ϵ 较大，agent 主要随机探索；训练后期 ϵ 衰减，agent 更多使用当前 learned Q function 的 greedy action。

2.4 DQN TODO 逐行注释：target network update

下面代码对应 DQN 的 update 函数。

```
# 从 replay buffer 中采样一个 batch, 并调用 critic update
loss = self.update_critic(observations, actions, rewards, next_observations, dones)

# 记录已经进行了多少次参数更新
self.num_param_updates += 1

# 每隔 target_update_freq 次更新, 把 online critic 参数复制给 target critic
if self.num_param_updates % self.target_update_freq == 0:
    # hard update: target 网络直接同步 online 网络
    self.target_critic.load_state_dict(self.critic.state_dict())
```

target network 的作用是让 TD target 变化慢一些。如果直接用 online network 自己生成 target，网络会一边追自己的预测一边更新，很容易震荡。

2.5 DQN TODO 逐行注释：run script 环境交互

下面代码对应 `run_hw3_dqn.py` 中训练循环的核心 TODO。

```
# 如果还没到 learning_starts, 或者 epsilon 探索触发, 则随机动作
if t < learning_starts or np.random.rand() < epsilon:
    action = env.action_space.sample()
else:
    # 否则调用 agent 的 greedy/epsilon-greedy 动作选择
    action = agent.get_action(obs, epsilon)

# 在环境中执行动作, 得到下一状态、奖励、done 标志和额外信息
next_obs, reward, done, info = env.step(action)

# 判断是否是真正终止; TimeLimit 截断不应作为 TD target 的 done
real_done = done and not info.get("TimeLimit.truncated", False)

# 将 transition 存入 replay buffer
replay_buffer.insert(obs, action, reward, next_obs, real_done)

# 如果 episode 结束, 则 reset 环境, 并通知 replay buffer 新轨迹开始
if done:
    obs = env.reset()
    replay_buffer.on_reset(obs)
else:
    # 否则继续推进当前轨迹
    obs = next_obs

# 到达训练起点后, 每隔 train_freq 步训练一次 critic
if t > learning_starts and t % train_freq == 0:
```

```
batch = replay_buffer.sample(batch_size)
agent.update(batch)
```

这里最容易错的是 `done` 与 `TimeLimit.truncated`。如果只是因为达到最大步数而被截断，真实 MDP 并没有终止，因此 TD target 中仍应该 bootstrap；否则会错误低估 value。

2.6 CartPole-v1 基础实验与学习率敏感性

本实验在 CartPole-v1 上测试四个 learning rate：10^{-5}、10^{-4}、10^{-3} 和 5×10^{-2}。结果如图 1。

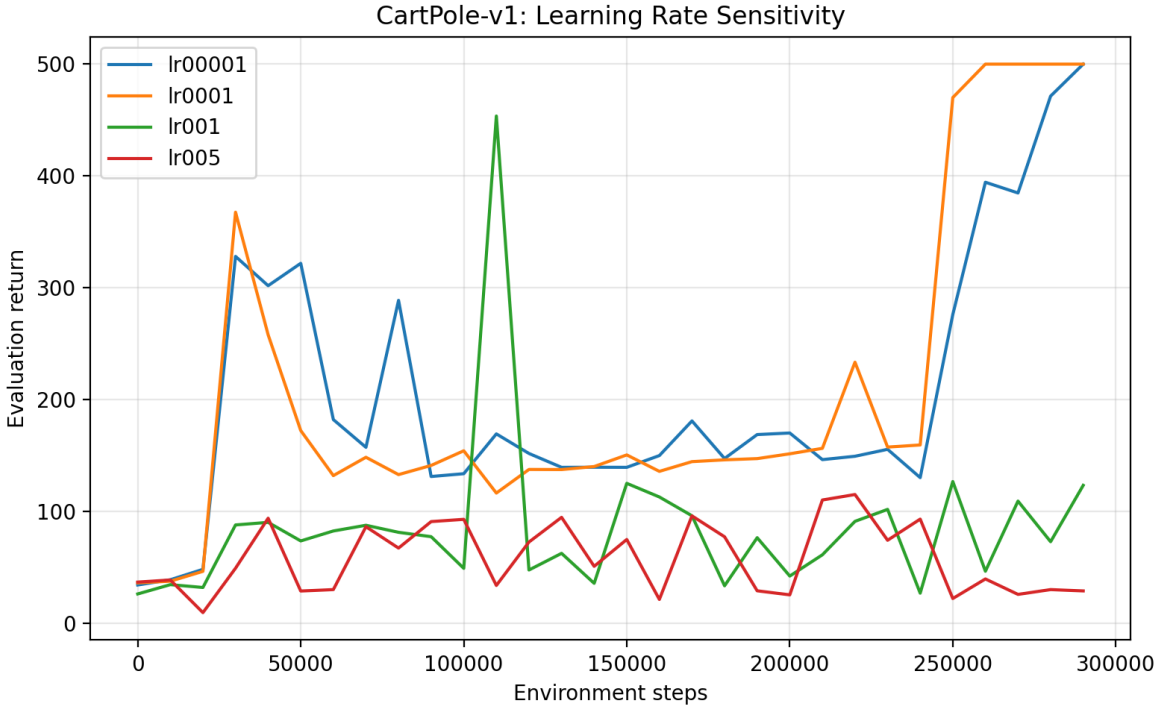
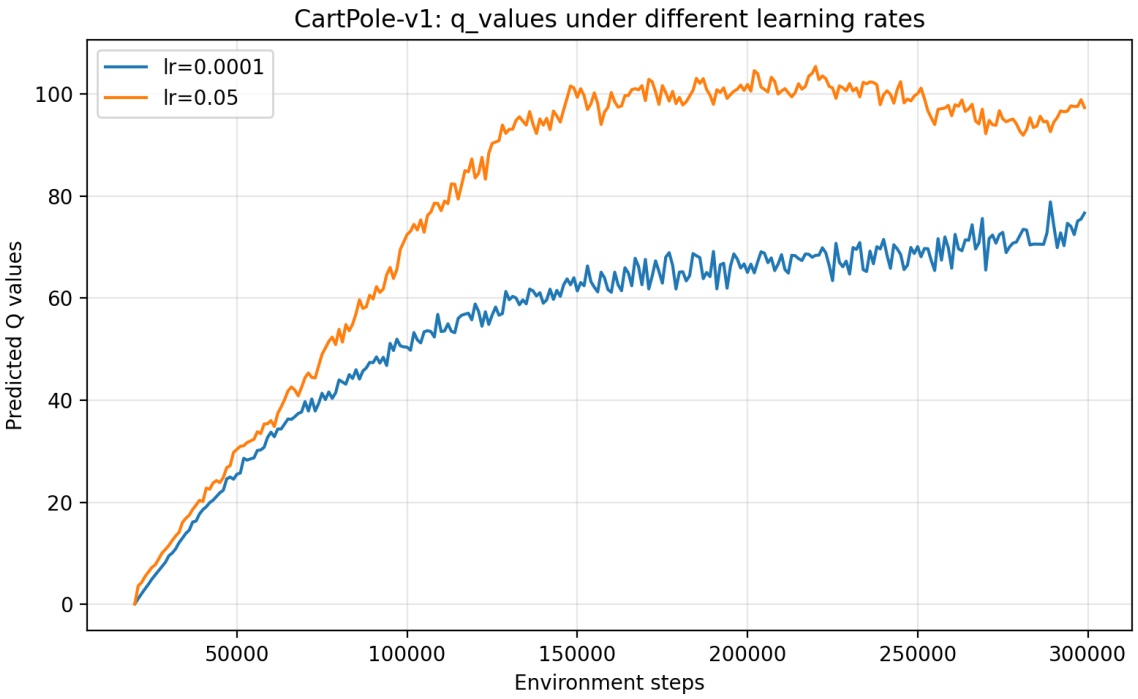


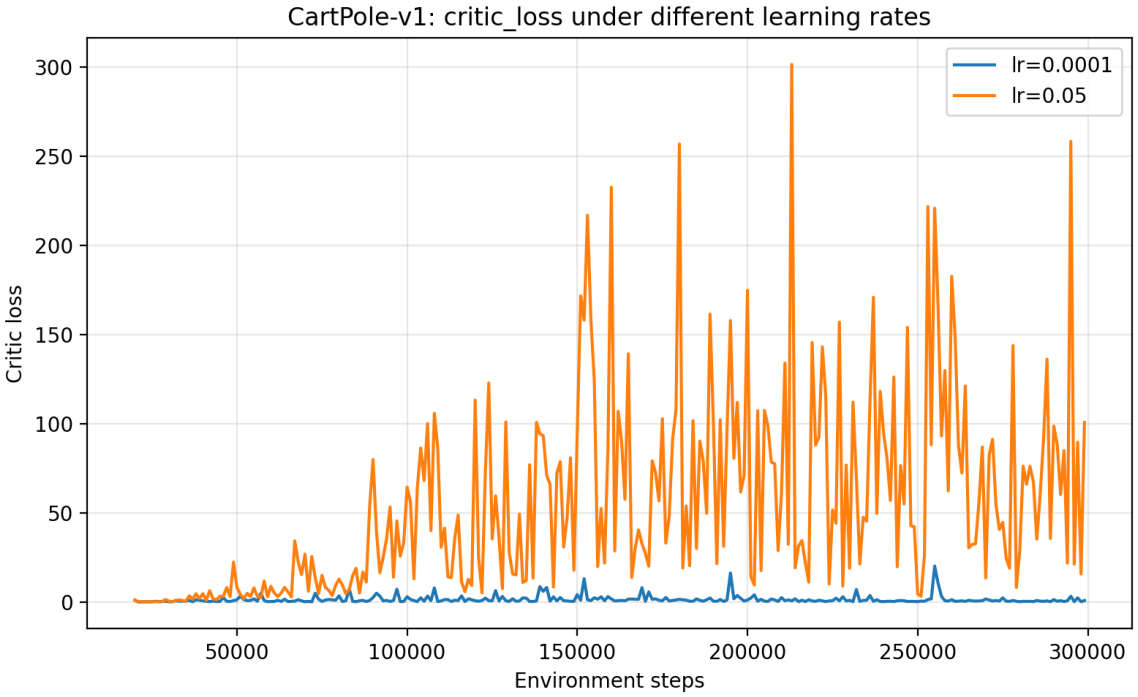
图 1: CartPole-v1 不同学习率下的 evaluation return。图例中的 lr00001、lr0001、lr001、lr005 分别表示 10^{-5}、10^{-4}、10^{-3}、5×10^{-2}。

从图中可以看到，较小学习率最终能够达到接近 500 的满分表现，而 5×10^{-2} 的学习率表现最差，长期停留在较低 return。这说明 DQN 对 learning rate 非常敏感。原因是 DQN 的 target 本身来自网络估计，如果学习率太大，一次更新会把 critic 推得过远，使下一轮 target 也随之剧烈变化，最终形成“追逐移动靶”的不稳定过程。

为了进一步分析 learning rate 过大时发生了什么，我对比了正常学习率 10^{-4} 与大学习率 5×10^{-2} 下的 predicted Q-values 和 critic loss，如图 2。



(a) predicted Q-values 对比。



(b) critic loss 对比。

图 2: CartPole-v1 中正常学习率与过大学习率对 Q 值和 critic loss 的影响。

数值上，10^{-4} 的最终 critic loss 约为 0.79，最大约为 20.15；而 5×10^{-2} 的最终 critic loss 约为 100.73，最大超过 300。同时，大学习率下 predicted Q-values 明显更高，最终约 97.36，最大约 105.42，而正常学习率最终约 76.67。这说明过大学习率会让 Q 值更容易被推高，critic error 也明显增大，对应理论部分中的 bootstrapping instability 与 overestimation bias。

2.7 Double DQN 实现与 LunarLander-v2 对比

Double DQN 的关键是：online network 用来选动作，target network 用来估值。其 target 为

$$a' = \arg\max_{a'} Q_\phi(s', a'), \qquad y = r + \gamma(1-d)Q_{\phi'}(s', a').$$

对应 TODO 实现已经在前面的 critic update 中体现，关键代码再次摘出如下。

```
# target critic 先计算所有动作的 target Q 值，后面用于估值
next_qa_values = self.target_critic(next_observations)

# Double DQN 中，动作选择来自 online critic，而不是 target critic
next_actions = self.critic(next_observations).argmax(dim=1)

# target critic 只负责评价 online critic 选出来的动作
next_q_values = next_qa_values.gather(1, next_actions.unsqueeze(1)).squeeze(1)

# 用分离后的 next_q_values 构造 TD target，减少 max over noisy estimates 的偏差
q_targets = rewards + self.gamma * (1 - dones.float()) * next_q_values
```

LunarLander-v2 上 vanilla DQN 与 Double DQN 的三 seed 对比如图 3。

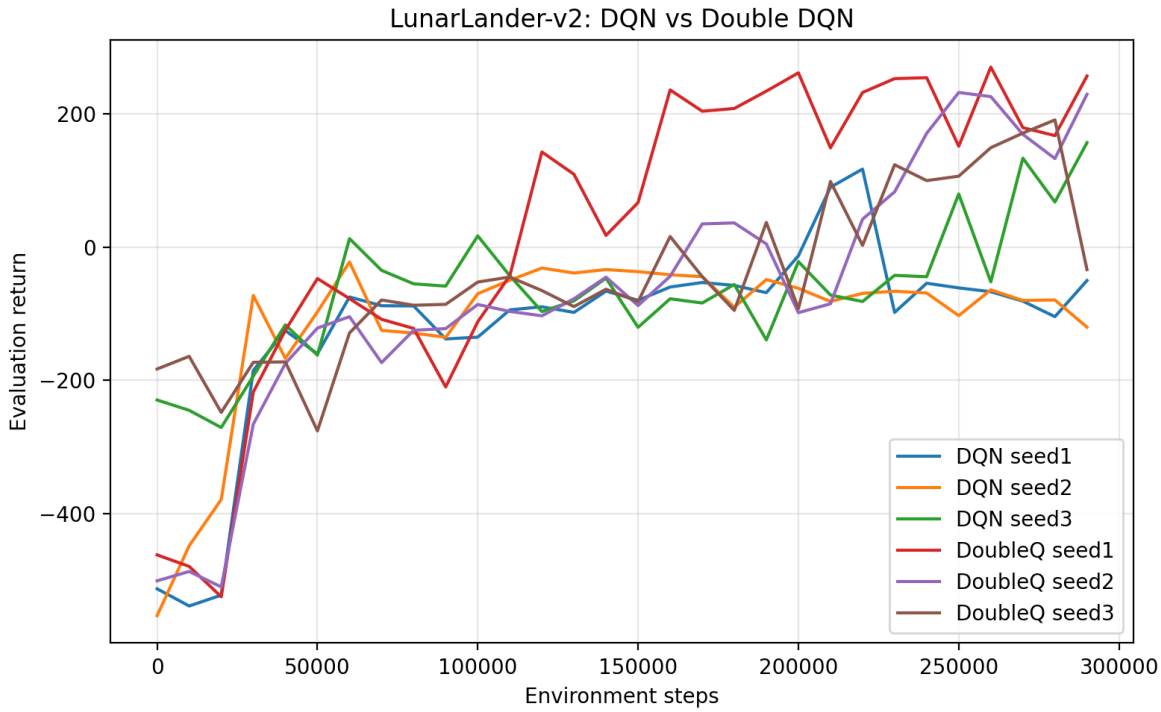


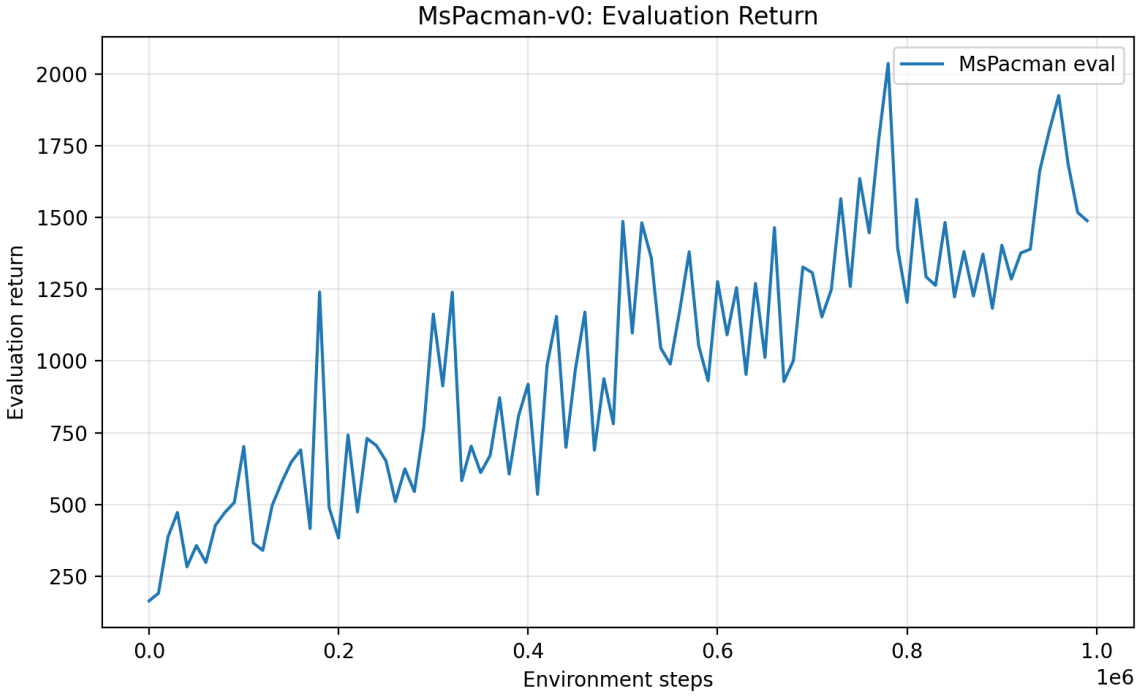
图 3: LunarLander-v2 上 vanilla DQN 与 Double DQN 的 evaluation return 对比。

实验中，vanilla DQN 三个 seed 的最终 return 约为 −50.4、−120.4、156.6；Double DQN 三个 seed 的最终 return 约为 256.8、229.3、−34.1。整体来看，Double DQN 更稳定，且有两个 seed 达到 200 分以上，而 vanilla DQN 有两个 seed 最终仍为负值。

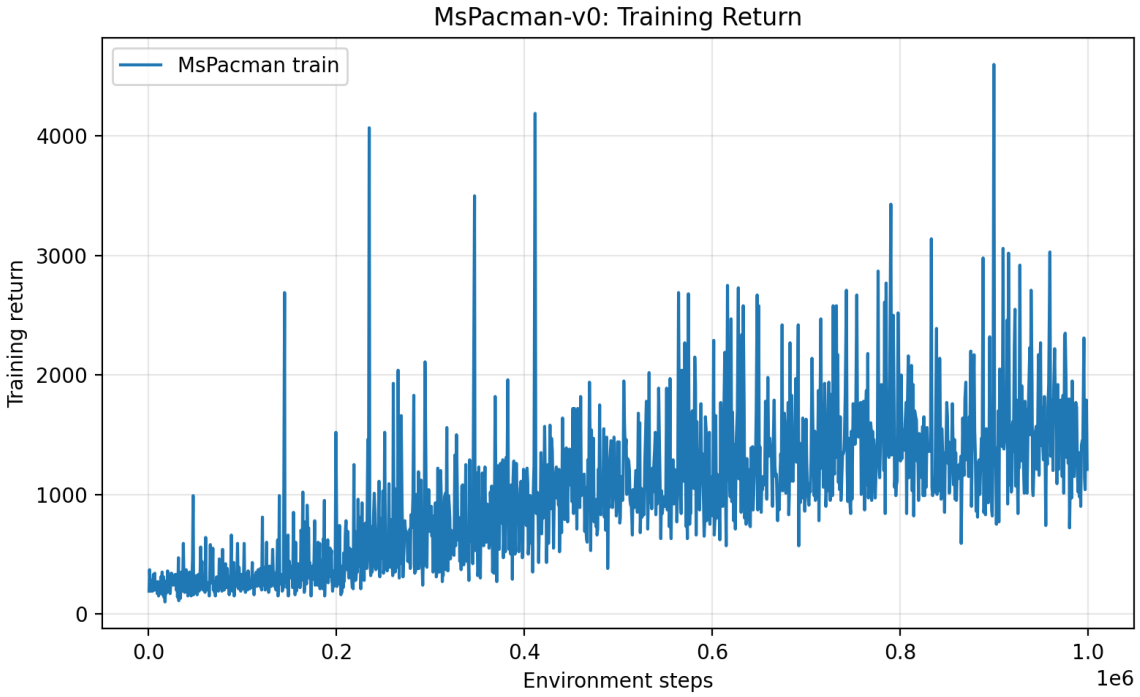
这个结果与理论预期一致。vanilla DQN 在 max backup 时容易选择被高估的动作，导致 target 偏大；Double DQN 则把动作选择和动作评价分离，减少了同一个噪声估计同时“选”和“估”的问题。因此 LunarLander 中 Double DQN 的曲线整体更高、更稳定。

2.8 MsPacman-v0 实验

MsPacman 是从像素输入学习的 Atari 任务，比 CartPole 和 LunarLander 更复杂。实验结果如图 4。



(a) evaluation return。



(b) training return。

图 4: MsPacman-v0 上 DQN 的 evaluation return 与 training return。

evaluation return 最终约为 1488，最大超过 2000，基本达到作业预期的 1500 左右。training return 曲线比 evaluation return 噪声大得多，这是正常现象。训练时 agent 采用 ϵ-greedy 策略，尤其在早期会故意随机探索，因此 training return 同时反映了“当前策略能力”和“探索噪声”；而 evaluation 通常使用更接近 greedy 的策略，因此更能体现已经学到的行为。

我自己的理解是，MsPacman 中 train/eval 差异非常像“考试”和“练习”的差异：训练时 agent 还在乱试、撞墙、吃亏，曲线自然很乱；evaluation 时让它认真发挥，曲线就更能体现真正学会了多少。

2.9 DQN 小结与思考

DQN 部分最重要的经验是：**Q-learning 的不稳定往往不是单一原因，而是 bootstrapping、max operator、function approximation 和 off-policy replay 共同作用的结果**。Target network、Double DQN、gradient clipping、learning rate scheduling 都是在给这个系统“减震”。如果学习率过大，即使其他 trick 都在，critic loss 仍然会爆炸式震荡，这也说明深度强化学习的训练稳定性很依赖细节。

3 Continuous Actions with Actor-Critic: SAC

3.1 SAC 实现总览

DQN 适合离散动作空间，因为 $\max_a Q(s,a)$ 可以通过枚举动作完成。连续动作空间中无法简单枚举所有动作，因此 SAC 同时学习 critic $Q_\phi(s,a)$ 和 actor $\pi_\theta(a|s)$。critic 负责评价动作，actor 负责产生高价值动作。

SAC 的 critic target 可写为

$$y = r + \gamma(1-d)\left[Q_{\phi'}(s',a') + \beta H(\pi(\cdot|s'))\right], \quad a' \sim \pi(\cdot|s').$$

若写成 log-prob 形式，entropy bonus 等价于 $-\beta \log \pi(a'|s')$。actor 的目标是最大化

$$\mathbb{E}_{a\sim\pi_\theta}\left[Q_\phi(s,a) - \beta \log \pi_\theta(a|s)\right].$$

也就是既要动作价值高，又要保持一定随机性用于探索。

3.2 SAC TODO 逐行注释：run script 交互与 replay buffer

下面代码对应 `run_hw3_sac.py` 的核心 TODO。

```
# 在随机探索阶段，直接从连续动作空间采样动作
if global_step < random_steps:
    action = env.action_space.sample()
else:
    # 之后使用 actor 输出的策略动作；通常训练时带随机采样，评估时可用均值动作
    action = agent.get_action(obs)

# 与环境交互，获得下一状态、奖励、done 标志和 info
next_obs, reward, done, info = env.step(action)

# TimeLimit 截断不是 MDP 真终止，不能把它当成 TD target 的 done
real_done = done and not info.get("TimeLimit.truncated", False)

# 将 transition 存入 replay buffer，供 off-policy 更新重复使用
replay_buffer.insert(obs, action, reward, next_obs, real_done)

# episode 结束时 reset 环境，否则继续推进
if done:
    obs = env.reset()
    replay_buffer.on_reset(obs)
else:
    obs = next_obs

# 从 replay buffer 采样 batch，更新 critic 和 actor
batch = replay_buffer.sample(batch_size)
```

```
agent.update(batch)
```

SAC 与 DQN 一样是 off-policy 方法，关键优势是 replay buffer 中旧数据可以被重复利用，因此 sample efficiency 通常高于 on-policy policy gradient。

3.3 SAC TODO 逐行注释：critic target 与 entropy bonus

下面代码对应 `soft_actor_critic.py` 中 critic update 的核心。

```
# 用当前 actor 在下一状态 s' 上构造动作分布
next_action_dist = self.actor(next_observations)

# 使用 rsample 采样下一动作；rsample 支持 reparameterization，可保留梯度路径
next_actions = next_action_dist.rsample()

# 计算采样动作的 log probability，连续动作通常需要对 action 维度求和
next_log_probs = next_action_dist.log_prob(next_actions).sum(dim=-1, keepdim=True)

# entropy 近似为 -log pi(a|s)，用于鼓励策略保持随机性
next_entropy = -next_log_probs

# 用 target critic 计算下一状态动作的 Q backup
next_q = self.q_backup_strategy(next_observations, next_actions)

# SAC target：reward + gamma * (Q + beta * entropy)，终止状态不 bootstrap
target_q = rewards + self.gamma * (1 - dones.float()) * (next_q + self.temperature *
↪  next_entropy)

# 当前 critic 对真实 batch 中 (s,a) 的估计
current_q = critic(observations, actions)

# critic 通过 MSE 拟合 target；target detach，避免梯度流入 actor 或 target critic
critic_loss = torch.nn.functional.mse_loss(current_q, target_q.detach())

# 清空 critic optimizer 中的旧梯度
critic_optimizer.zero_grad()

# 反向传播 critic loss
critic_loss.backward()

# 更新 critic 参数
critic_optimizer.step()
```

这里的关键理解是：critic 不只学习 “未来 reward”，还学习 “未来策略熵”。如果策略在某些状态还不确定，entropy bonus 会鼓励保留探索，这也是 SAC 比普通 actor-critic 更稳定的重要原因。

3.4 SAC TODO 逐行注释：target network soft/hard update

SAC 中 target critic 可以 hard update，也可以 Polyak soft update。

```
# hard update：每隔固定步数，把 online critic 参数完整复制到 target critic
if self.target_update_period is not None and step % self.target_update_period == 0:
    target_critic.load_state_dict(critic.state_dict())

# soft update：每一步都让 target 参数向 online 参数移动一点点
for target_param, param in zip(target_critic.parameters(), critic.parameters()):
    # tau 很小，因此 target 网络变化慢，target 更平滑
    target_param.data.copy_(target_param.data + self.tau * (param.data -
    ↪  target_param.data))
```

DQN 常用 hard update；SAC 更常见的是 soft update。soft update 的好处是 target 连续平滑变化，减少 critic target 的震荡。

3.5 SAC TODO 逐行注释：entropy 计算

```
# 从 actor distribution 中采样若干动作，用于近似 entropy
actions = action_distribution.rsample(sample_shape=(num_samples,))

# 计算每个动作的 log probability，并对动作维度求和
log_probs = action_distribution.log_prob(actions).sum(dim=-1)

# entropy 定义为 E[-log pi(a|s)]，用采样平均近似
entropy = -log_probs.mean()
```

Pendulum sanity 中，1 维动作空间若近似均匀分布在 $[-1, 1]$，最大 entropy 约为 $\log 2 \approx 0.69$。如果 entropy 长期偏离很大，通常说明 log probability 或动作 squashing 的处理有问题。

3.6 SAC TODO 逐行注释：REINFORCE actor loss

REINFORCE 估计的是

$$\nabla_\theta \mathbb{E}_{a\sim\pi_\theta}[Q_\phi(s,a)] = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a|s) Q_\phi(s,a)\right].$$

对应实现如下。

```
# 对 batch 中每个 state 构造 actor 的动作分布
action_dist = self.actor(observations)

# 从策略中采样动作；REINFORCE 不需要对采样动作本身反传，所以用 sample 即可
sampled_actions = action_dist.sample((self.num_actor_samples,))
```

```
# 计算每个采样动作的 log probability
log_probs = action_dist.log_prob(sampled_actions).sum(dim=-1)

# 为了让每个 state 对应多个 sampled action, 需要把 observations 扩展/重复
expanded_obs = observations.repeat(self.num_actor_samples, 1)

# critic 评价这些策略采样动作的 Q 值
q_values = self.critic(expanded_obs, sampled_actions.reshape(-1, action_dim))

# Q 值在 REINFORCE 中作为权重使用, 不应让 actor loss 反传进 critic
q_values = q_values.detach().reshape(self.num_actor_samples, -1)

# 最大化 E[log pi(a|s) * Q(s,a)], 写成最小化时前面加负号
policy_gradient_loss = -(log_probs * q_values).mean()

# entropy bonus: 最大化 entropy, 写成 loss 时也是负号
entropy_loss = -self.temperature * self.entropy(action_dist)

# 总 actor loss = REINFORCE loss + entropy loss
actor_loss = policy_gradient_loss + entropy_loss
```

REINFORCE 的问题是高方差。增加 num_actor_samples 可以降低方差，但计算量也会增加，且在高维动作空间中仍然不够高效。

3.7 SAC TODO 逐行注释：reparameterized actor loss

Reparameterization trick 将动作写为

$$a = \mu_\theta(s) + \sigma_\theta(s)\epsilon, \quad \epsilon \sim \mathcal{N}(0, I),$$

于是可以直接对 $Q(s, a)$ 关于 actor 参数求梯度。

```
# actor 输出当前状态下的动作分布
action_dist = self.actor(observations)

# rsample 进行可重参数化采样, 梯度可以从 action 回传到 actor 参数
actions = action_dist.rsample()

# 计算动作 log probability, 用于 entropy regularization
log_probs = action_dist.log_prob(actions).sum(dim=-1, keepdim=True)

# critic 评价 reparameterized action 的 Q 值
q_values = self.critic(observations, actions)

# SAC actor objective: maximize Q(s,a) - beta * log pi(a|s)
```

```
actor_objective = q_values - self.temperature * log_probs

# PyTorch 默认最小化 loss，所以对 objective 取负号
actor_loss = -actor_objective.mean()
```

相比 REINFORCE，reparameterization 的梯度通常方差低很多，因为梯度直接穿过动作采样路径，而不是只靠 $\nabla \log \pi$ 和 noisy Q-value 的乘积。

3.8 SAC TODO 逐行注释：single-Q, double-Q, clipped-Q backup

Hopper 实验需要实现不同 target backup。

```
# 分别用每个 target critic 计算 Q_i(s', a')
next_qs = [target_critic(next_observations, next_actions) for target_critic in
↪  self.target_critics]

# single-Q：只使用第一个 target critic
if self.target_critic_backup_type == "single":
    q_backup = next_qs[0]

# double-Q：两个 critic 交叉作为彼此的 target，减少自举自评估带来的偏差
elif self.target_critic_backup_type == "doubleq":
    q_backup_for_critic_1 = next_qs[1]
    q_backup_for_critic_2 = next_qs[0]
    q_backup = [q_backup_for_critic_1, q_backup_for_critic_2]

# clipped double-Q：取两个 target critic 的较小值，主动保守估计
elif self.target_critic_backup_type == "min":
    q_backup = torch.min(next_qs[0], next_qs[1])
```

clipped double-Q 的思想非常直接：如果两个 critic 都有估计误差，取最大容易高估，取最小则偏保守。保守估计可能牺牲一点乐观探索，但能显著缓解 overestimation bias，在连续控制中通常更稳定。

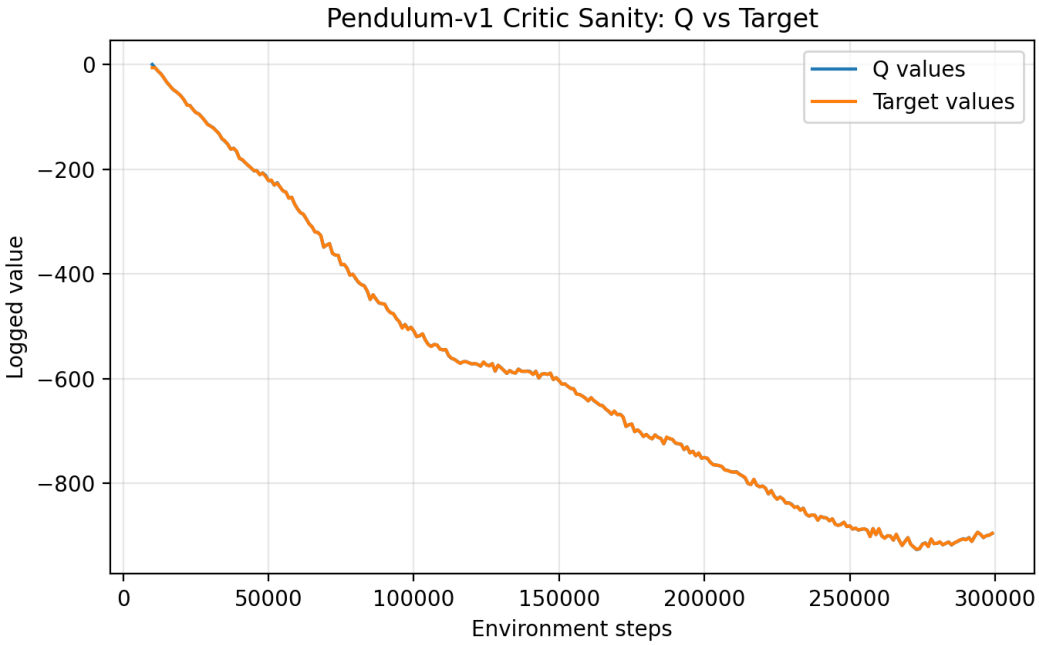
3.9 SAC sanity check：Bootstrapping、Entropy 与 InvertedPendulum-v4

它能证明从 bootstrapping 往后的SAC 代码不是只写了公式，而是真的能在简单连续控制环境上跑通。

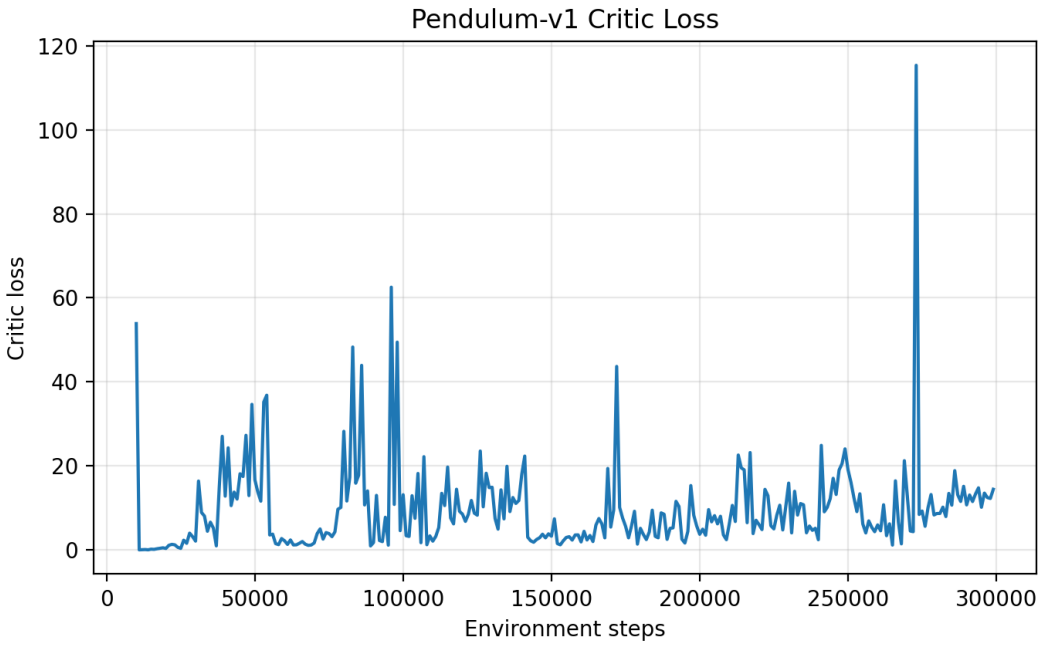
3.9.1 Pendulum-v1：bootstrapping critic sanity check

作业在 SAC 的 bootstrapping 部分要求先用 `sanity_pendulum.yaml` 检查 critic 是否能正常训练。这个实验的目的不是让 Pendulum 取得高回报，因为此时重点还在 critic target 是否稳定；更关键的是看 Q values 与 target values 是否进入同一量级，critic loss 是否没有发散。

当前日志中，Pendulum-v1 的 `q_values` 最终约为 -895.77，`target_values` 最终约为 -895.58，二者已经非常接近；critic loss 最终约为 14.47，没有出现 Q 值向负无穷发散或长期停留在 0 附近的问题。这个结果说明 bootstrapped critic update、target critic 以及 replay buffer 采样这几部分至少已经形成了正确闭环。



(a) Pendulum-v1 中 logged Q-values 与 target values。



(b) Pendulum-v1 中 critic loss。

图 5: Pendulum-v1 bootstrapping sanity check。该实验主要用于验证 critic target 是否正常，而不是作为最终性能指标。

3.9.2 Entropy sanity check

作业还要求实现 entropy，并把 entropy term 加入 critic target 和 actor loss。理论上，一维动作空间若接近均匀分布在 $[-1, 1]$，最大熵约为

$$H(U[-1,1]) = -\log \frac{1}{2} = \log 2 \approx 0.69.$$

当前日志中，Pendulum-v1 与 InvertedPendulum-v4 在训练早期的 entropy 都接近 $0.64 \sim 0.67$，和 $\log 2$ 的 sanity check 预期是同一量级。这说明 entropy 计算的符号和 action 维度求和逻辑基本是对的。后期 entropy 下降并不奇怪，因为 actor 开始为了取得更高 return 而变得更确定；报告中更重要的是说明早期 sanity check 没有明显异常。

3.9.3 InvertedPendulum-v4：REINFORCE actor sanity check

作业在 REINFORCE actor 部分明确要求用 `sanity_invertedpendulum_reinforce.yaml` 训练 InvertedPendulum-v4，并期望 reward 接近 1000。当前代码包中存在对应日志与 TensorBoard event。实验结果显示，REINFORCE 版本最终 evaluation return 约为 940.0，训练过程中最高达到 1000.0。由于 InvertedPendulum-v4 的满分通常对应智能体保持倒立直到 episode 结束，因此该结果说明 REINFORCE actor loss、actor update 和 critic feedback 已经可以共同学出稳定策略。

3.9.4 InvertedPendulum-v4：reparameterization actor sanity check

作业在 reparameterization 部分同样要求用 `sanity_invertedpendulum_reparametrize.yaml` 检查是否能 solve InvertedPendulum-v4。当前日志中，reparameterization 版本最终 evaluation return 约为 996.1，最高同样达到 1000.0。相比 REINFORCE，reparameterization 的最终结果更接近满分，说明可重参数化梯度在这个简单连续控制任务上更稳定，也和后面 HalfCheetah-v4 中 reparameterization 明显优于 REINFORCE 的现象一致。

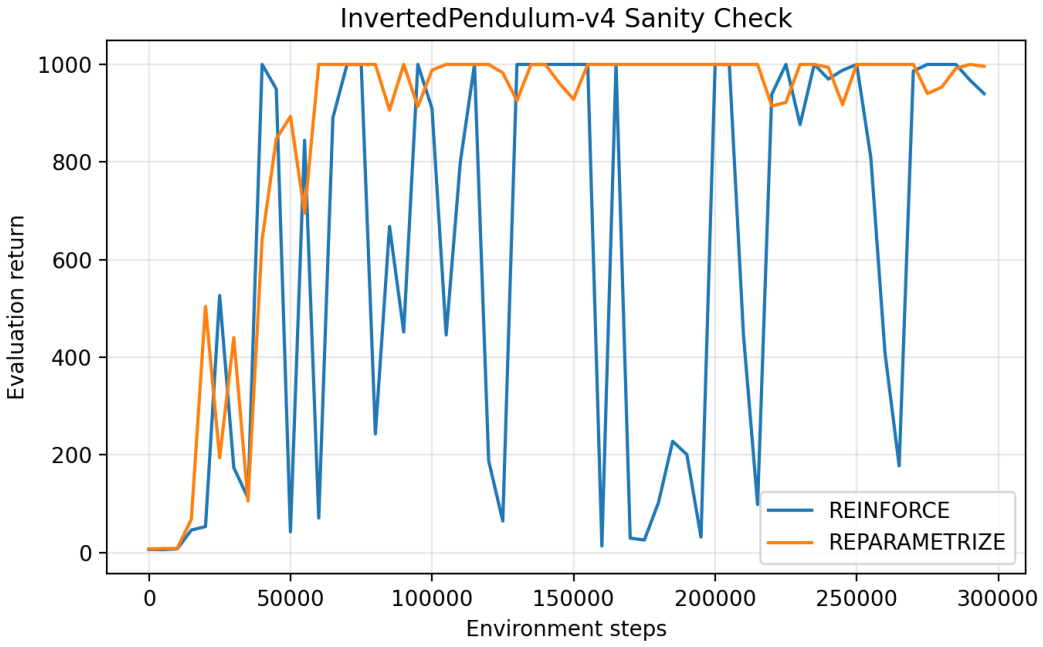


图 6: InvertedPendulum-v4 sanity check：REINFORCE 与 REPARAMETRIZE 的 evaluation return。两者都能接近或达到 1000，说明 actor 更新部分已经跑通。

这些 sanity check 放在 HalfCheetah 正式实验之前最合适。
逻辑顺序是：先验证 critic bootstrapping 没有明显发散，再验证 entropy 的量级合理，然后分别验证 REINFORCE actor 与 reparameterized actor 都能在 InvertedPendulum-v4 上学会控制，最后再进入 HalfCheetah、 Hopper 和 Humanoid 这些更正式、更难的 deliverable 实验。

3.10 HalfCheetah-v4：actor gradient estimator 对比

HalfCheetah-v4 上三种 actor gradient estimator 的结果如图 5。

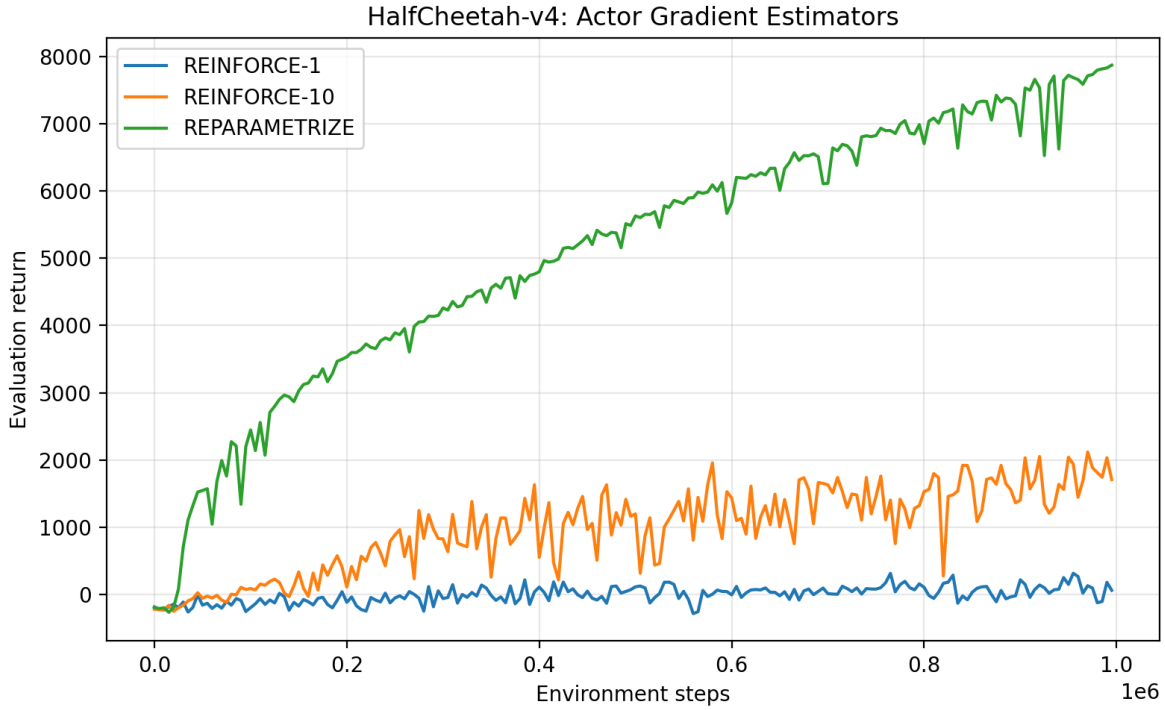


图 5: HalfCheetah-v4 上 REINFORCE-1、REINFORCE-10 与 REPARAMETRIZE 的 evaluation return。

最终 return 约为：REINFORCE-1 为 62.8，REINFORCE-10 为 1709.5，REPARAMETRIZE 为 7871.1。结果差距非常明显：reparameterization 最好，REINFORCE-10 次之，REINFORCE-1 最差。

REINFORCE-10 明显好于 REINFORCE-1，说明增加采样数确实能降低 policy gradient 的估计方差。但即便采样 10 次，REINFORCE 仍远不如 reparameterization。这说明在高维连续动作空间中，仅靠 score function estimator 很难得到稳定低方差梯度；而 reparameterization 可以让梯度直接通过动作样本传回 actor 参数，因此学习效率高得多。

3.11 Hopper-v4：target backup strategy 对比

Hopper-v4 上 single-Q、double-Q 和 clipped-Q 的 evaluation return 如图 6，Q-value estimates 如图 7。



图 6: Hopper-v4 上不同 target backup strategy 的 evaluation return。

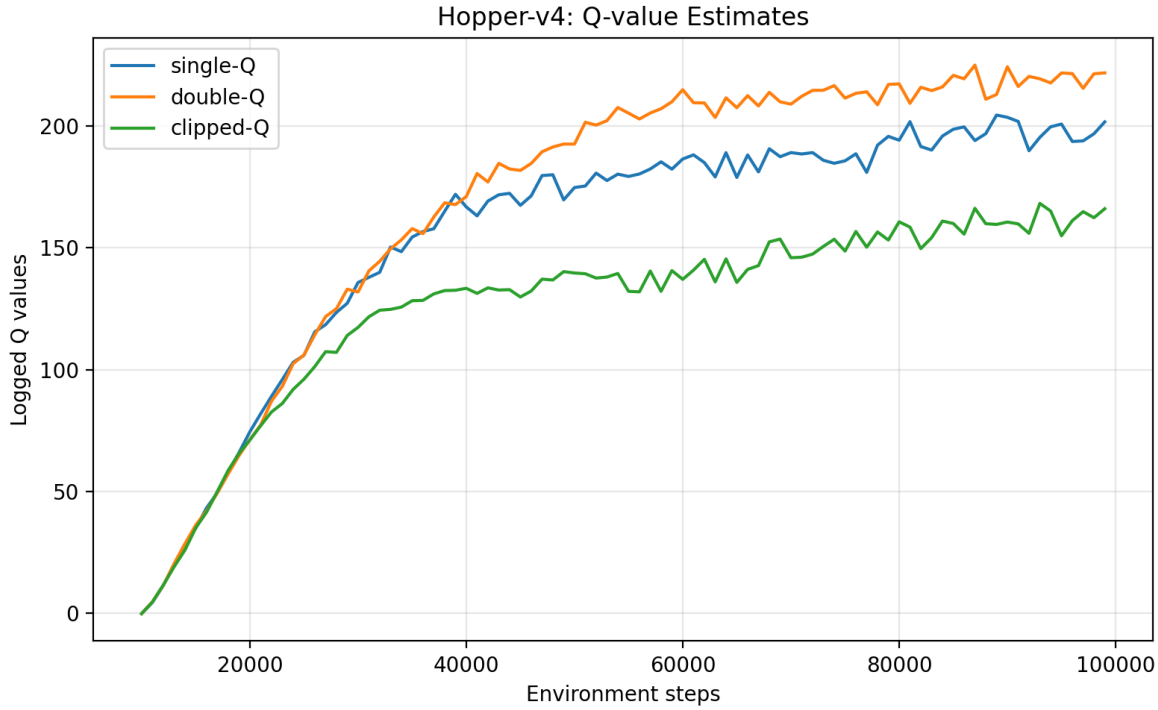


图 7: Hopper-v4 上不同 target backup strategy 的 logged Q-values。

最终 evaluation return 约为：single-Q 317.5，double-Q 366.6，clipped-Q 536.5。最终 logged Q-values 约为：single-Q 201.7，double-Q 221.7，clipped-Q 166.0。最有意思的是，clipped-Q 的 Q 值最低，但真实 return 最高。

这正好说明了 clipped double-Q 的作用：它不是追求让 Q 值看起来更大，而是让 target 更保守、更可靠。single-Q 和 double-Q 的 Q estimate 更高，但 policy 表现反而不如 clipped-Q，说明它们更可能受到 overestimation bias 影响。clipped-Q 通过取两个 target critic 的最小值，牺牲一部分乐观估计，换来了更稳定的训练目标。

3.12 Humanoid-v4：最佳配置迁移

由于 Hopper 上 clipped-Q 表现最好，因此我选择 clipped double-Q 作为 Humanoid-v4 的配置，并按作业要求将训练截断到 500K environment steps。结果如图 8。

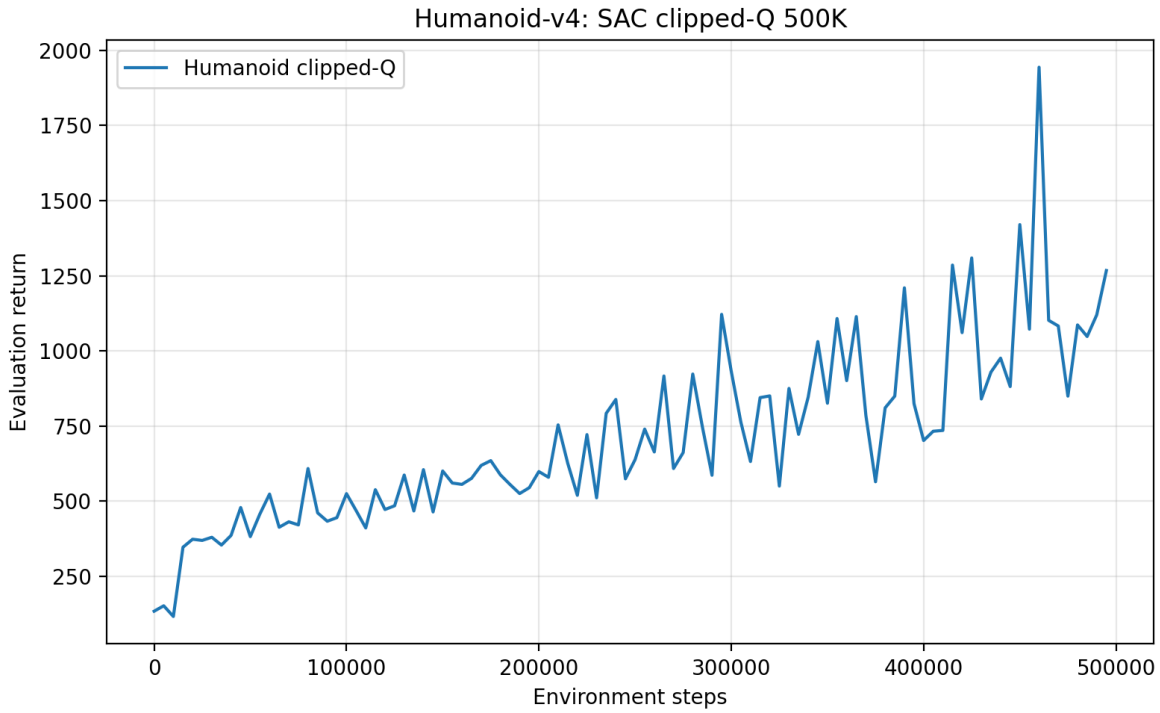


图 8: Humanoid-v4 上 clipped-Q SAC 训练 500K steps 的 evaluation return。

Humanoid 的最终 evaluation return 约为 1267.3，最高约为 1943.6。曲线整体上升，但噪声也很大。这是合理的：Humanoid 的状态维度和动作维度都更高，控制目标也更复杂，探索难度远大于 Hopper 和 HalfCheetah。

从 sample efficiency 的角度看，SAC 作为 off-policy 方法可以重复利用 replay buffer 中的数据，因此在 500K steps 内已经能得到明显提升。相比 on-policy policy gradient，off-policy actor-critic 通常更省样本；但代价是实现更复杂，也更依赖 critic 稳定性。我的理解是：SAC 的强大不是来自某一个单独 trick，而是 “entropy exploration + replay buffer + target critic + clipped double-Q + reparameterization” 共同形成了一个相对稳的训练系统。

4 整体讨论与个人思考

4.1 从理论到实验的对应关系

HW3 的理论题和实验题其实是连在一起的。理论部分说明 max over noisy Q-values 会导致 bias；实验中，DQN 的学习率过大时 predicted Q-values 更高、critic loss 更大，Double DQN 则提升了 LunarLander 稳定性。SAC 的 Hopper 实验进一步说明，在连续动作 actor-critic 中，critic 的 overestimation bias 仍然存在，而且 clipped double-Q 能有效缓解。

所以这次作业最重要的主线不是“把算法跑起来”，而是理解：**critic 一旦不稳，整个 RL 系统都会跟着不稳**。因为 policy improvement 依赖 critic，如果 critic 高估错误动作，actor 会被引向错误方向；然后 replay buffer 里又会出现更多由错误策略产生的数据，错误可能继续放大。

4.2 为什么 reparameterization 如此重要

HalfCheetah 的结果让我印象最深。REINFORCE-1 几乎没有学起来，REINFORCE-10 有进步，但 reparameterization 直接把 return 拉到 7000 以上。这说明连续控制中，policy gradient 的方差是非常现实的问题。

REINFORCE 的梯度像是在说：“这个动作最后 Q 值高，所以以后增加它的 log probability。”但这个估计非常吵，尤其动作维度高时，一个动作样本很难代表整个分布。Reparameterization 则像是在说：“动作是 actor 参数的可微函数，我可以直接沿着让 Q 变大的方向微调动作分布。”这条梯度路径明显更直接，因此训练效率更高。

4.3 我对 DQN 与 SAC 的比较

DQN 的结构相对简单：一个 Q-network、一个 target Q-network、一个 replay buffer。但因为它要对动作取 max，所以更适合离散动作空间。SAC 则显式学习 actor，因此可以处理连续动作，但它需要维护 actor、critic、target critic、entropy regularization 等多个组件。

我自己的理解是：DQN 像是“先给每个动作打分，再选最高分”；SAC 像是“直接训练一个会出动作的决策器，同时用 critic 告诉它动作好不好”。前者简单直接，后者复杂但更适合机器人控制这类连续动作任务。

5 结论

本次作业完整实现并实验验证了 DQN、Double DQN 和 SAC。主要结论如下：

1. Bellman max backup 会产生 overestimation bias，即使单个 $\hat{Q}$ 是无偏估计，$\max_a \hat{Q}$ 也不一定无偏。
2. DQN 对学习率非常敏感。CartPole 中 5×10^{-2} 的学习率导致 critic loss 大幅上升，最终策略表现很差。

3. Double DQN 在 LunarLander 上整体优于 vanilla DQN，说明分离动作选择和动作评价能缓解过估计。

4. MsPacman 实验表明 DQN 可以扩展到像素输入 Atari 任务，最终 evaluation return 接近作业预期。

5. SAC 中 reparameterized actor gradient 明显优于 REINFORCE，尤其在 HalfCheetah 这类连续控制任务中优势巨大。

6. Hopper 实验中 clipped double-Q 的 Q 值最低但 return 最高，说明保守 critic target 反而能带来更好的策略。

7. Humanoid 500K 实验显示，使用 Hopper 中表现最好的 clipped-Q 配置，在更高维连续控制任务中也能取得明显进步。

总体来说，HW3 让我更清楚地看到深度强化学习的核心矛盾：agent 需要根据自己的估计来改进策略，但这些估计本身又来自有限数据和函数近似。DQN 和 SAC 的许多设计，本质上都是在降低这种“自己教自己”带来的偏差、方差和不稳定性。

A 实验结果数值摘要

表 2: 主要实验最终结果摘要

实验	指标	结果
CartPole, lr=10^{-5}	final eval return	500.0
CartPole, lr=10^{-4}	final eval return	500.0
CartPole, lr=10^{-3}	final eval return	123.4
CartPole, lr=5×10^{-2}	final eval return	29.0
CartPole, lr=10^{-4}	final critic loss	0.79
CartPole, lr=5×10^{-2}	final critic loss	100.73
LunarLander DQN seed1	final eval return	-50.4
LunarLander DQN seed2	final eval return	-120.4
LunarLander DQN seed3	final eval return	156.6
LunarLander DoubleQ seed1	final eval return	256.8
LunarLander DoubleQ seed2	final eval return	229.3
LunarLander DoubleQ seed3	final eval return	-34.1
MsPacman	final eval return	1488.0
MsPacman	max eval return	2036.0
HalfCheetah REINFORCE-1	final eval return	62.8
HalfCheetah REINFORCE-10	final eval return	1709.5
HalfCheetah REPARAMETRIZE	final eval return	7871.1
Hopper single-Q	final eval return	317.5

实验	指标	结果
Hopper double-Q	final eval return	366.6
Hopper clipped-Q	final eval return	536.5
Hopper single-Q	final Q value	201.7
Hopper double-Q	final Q value	221.7
Hopper clipped-Q	final Q value	166.0
Humanoid clipped-Q 500K	final eval return	1267.3
Humanoid clipped-Q 500K	max eval return	1943.6